codeHive

DSA Arrays

The Foundation of Data Management

Module 01: Core Structures

What are Arrays?

An **array** is a collection of elements stored at contiguous memory locations. It is the most fundamental data structure used by countless algorithms to organize and process information.

In Python, the 'list' type acts as our array. Whether you are finding the lowest value or sorting numbers, the array is your primary tool.

Key Characteristics

- **Fixed/Dynamic Size:** Stores multiple values in one place.
- **Fast Access:** Access any element instantly if you know its position.
- **Contiguous:** Elements are placed right next to each other in memory.

Python Example

```python
my_array = [7, 12, 9, 4, 11]
```

Zero-Based Indexing

In modern programming (Python, Java, C), we start counting from **0**. This is known as 0-based indexing.

Accessing Data:

To get the first element (7), you use: `my_array[0]`

To get the fourth element (4), you use: `my_array[3]`

Algorithm: Finding the Lowest Value

The best way to understand arrays is to build an algorithm that interacts with one. Let's find the minimum value in a set.

The Pseudocode

```
1. Create a variable 'minVal' and set it to the first value of the array.
2. For each element in the array:
   a. If current element < minVal:
      Update 'minVal' to the current element.
3. Final 'minVal' is the lowest value correctly identified.
```

Why use Pseudocode?

Pseudocode is a bridge between human thought and computer code. It helps you focus on **logic** without getting distracted by the "syntax" (grammar) of a specific programming language.

Python Implementation

Now, we translate our logic into actual executable Python code.

```python
my_array = [7, 12, 9, 4, 11]

# Step 1: Initialize with first value
minVal = my_array[0]

# Step 2: Iterate through the collection
for i in my_array:
    # Step 3: Comparison logic
    if i < minVal:
        minVal = i

# Step 4: Output result
print('Lowest value:', minVal)
```

Step-by-Step Execution

- **Start:** `minVal` is 7.
- **Loop 1:** Compare 12 to 7. (No change)
- **Loop 2:** Compare 9 to 7. (No change)
- **Loop 3:** Compare 4 to 7. (Update: `minVal` is now 4)
- **Loop 4:** Compare 11 to 4. (No change)

Algorithm Complexity

In Computer Science, we measure how long an algorithm takes to run relative to the size of the data. This is called **Time Complexity**.

LINEAR TIME

O(n)

The time taken grows proportionally with the number of elements.

Why O(n)?

Because the algorithm must visit **every** element at least once to ensure it has seen the smallest value.

- If the array has 5 values, we do 5 checks.
- If the array has 1,000,000 values, we do 1,000,000 checks.

Knowledge Check

Exercise: Identifying Indices

Given the array: `vals = [10, 20, 30, 40, 50]`

How do we print the value **"10"**?

Answer: print(vals[0])

codeHive

End of Module 01 | Ready for Sorting Algorithms?